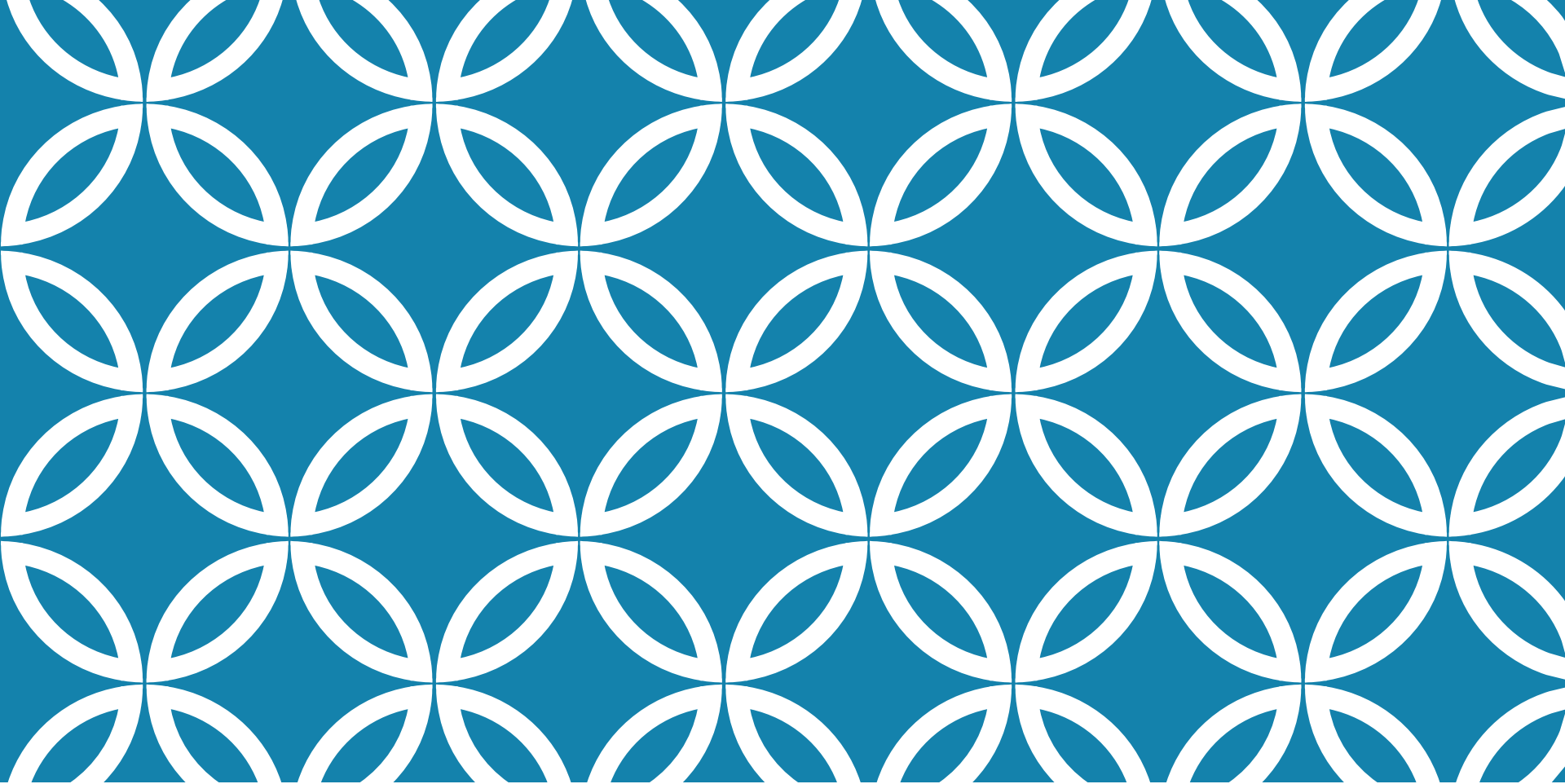




EVENT-DRIVEN ARCHITECTURE

Lecture 6

LAST TIME

Architectural Styles

- Microkernel
- Service-based
- Domain-driven design

CONTENT

Architectural Styles

- Event-driven
 - Broker
 - Mediator

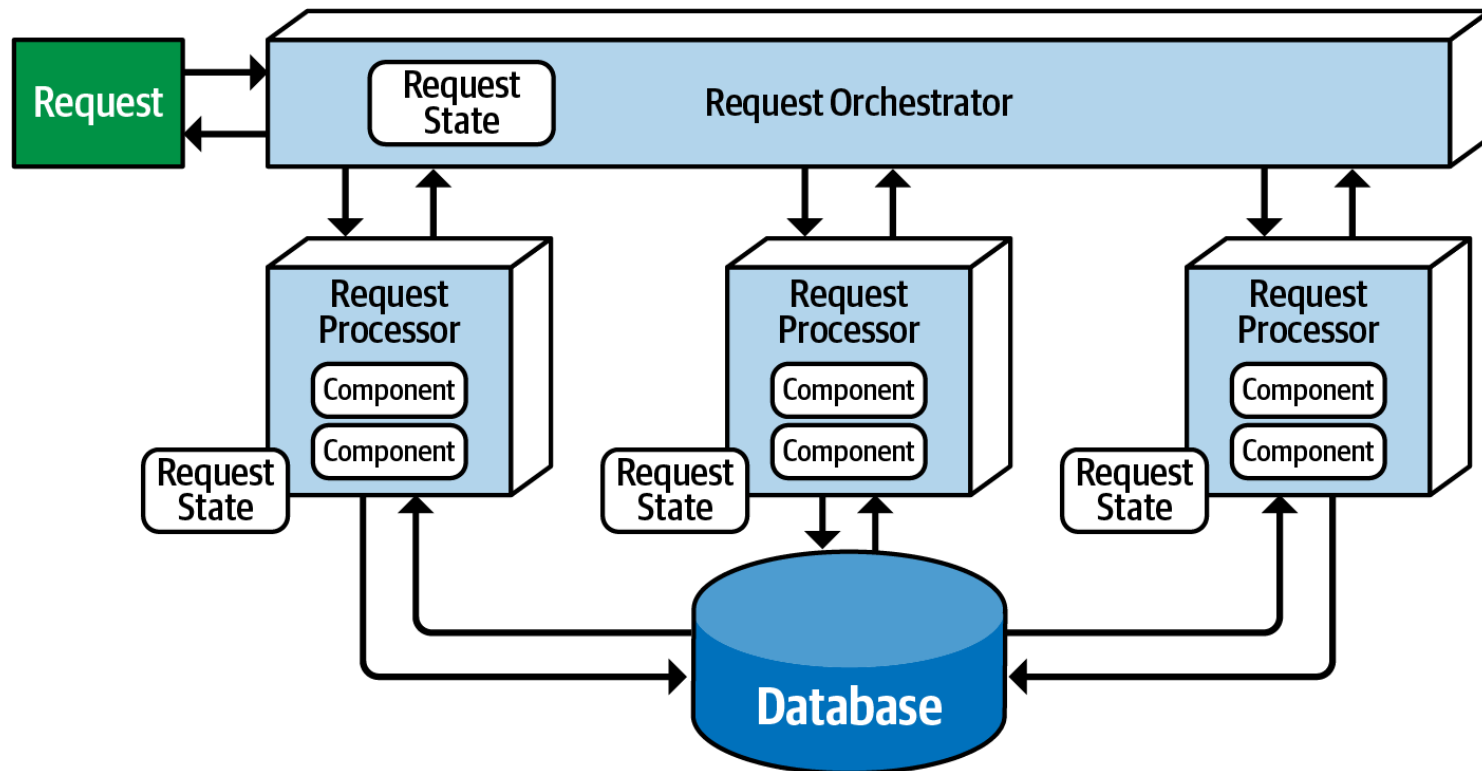
REFERENCES

Fundamentals of Software Architecture, Mark Richards,
Neal Ford, 2020 - Chapter 14

EVENT-DRIVEN ARCHITECTURE

Distributed, asynchronous architectural style

Event-based vs Request-based architectures



EVENT-DRIVEN TOPOLOGIES

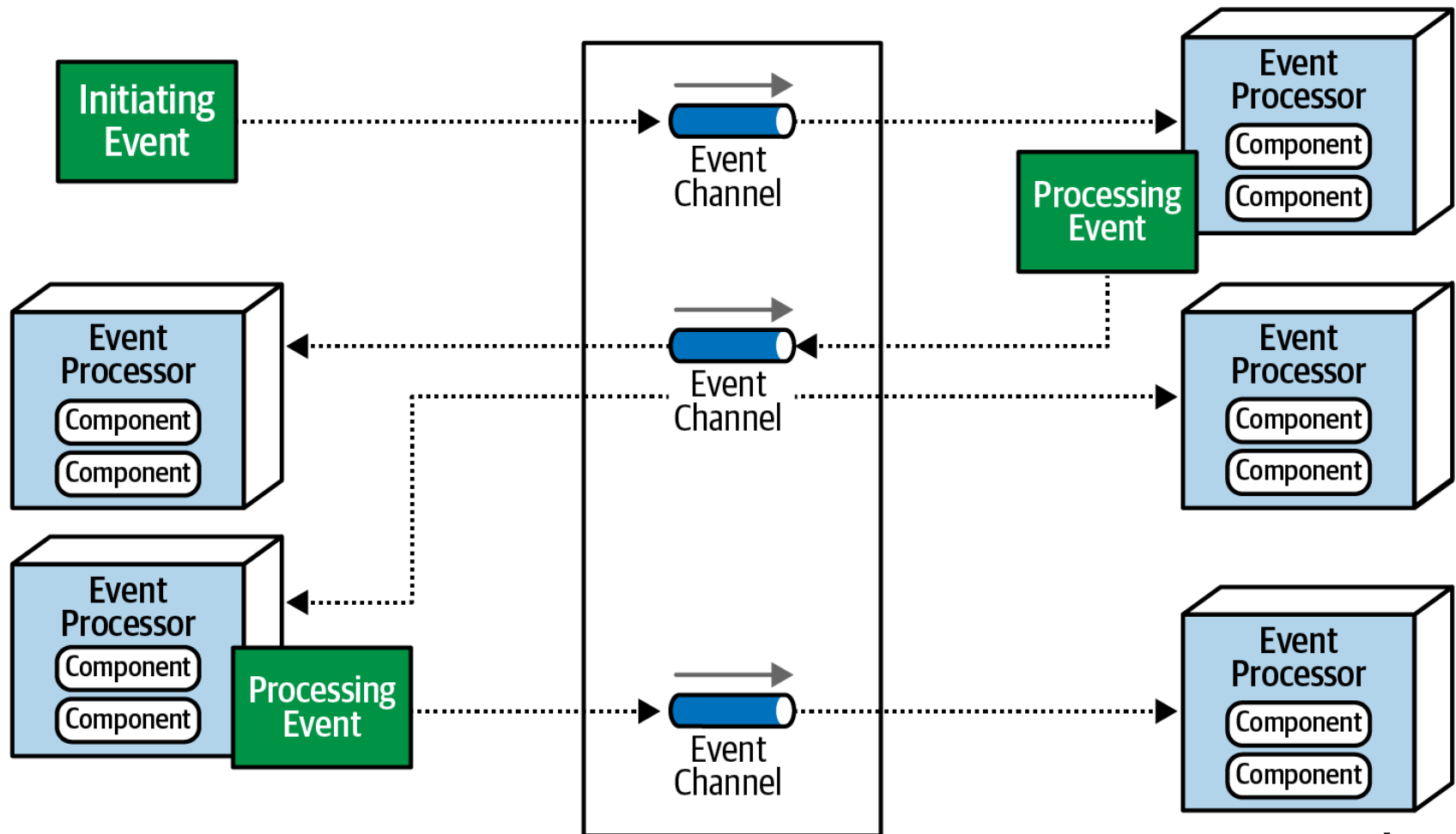
- **Broker**

- High degree of **responsiveness** and **dynamic control** over the processing of an event
- No central event mediator
- Message flow is distributed in a chain-like broadcasting fashion through a lightweight message broker (such as RabbitMQ, ActiveMQ, HornetQ, etc)
- Useful when you have a relatively simple event processing flow and you do not need central event orchestration and coordination

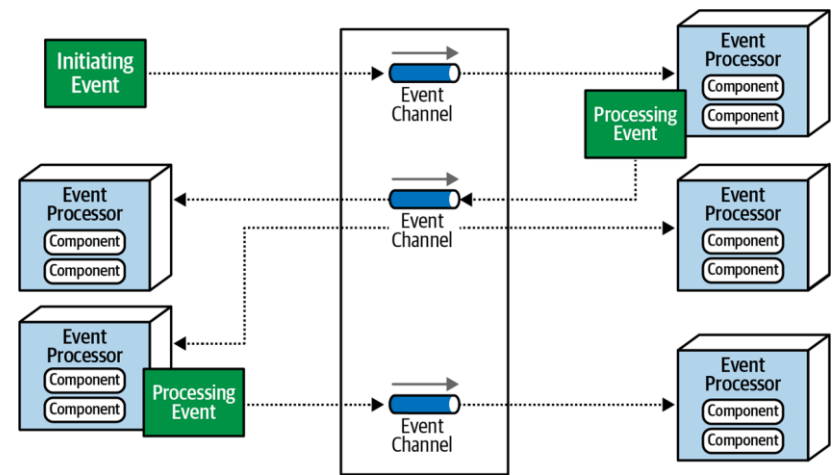
- **Mediator**

- Controls the workflow of an event process

BROKER TOPOLOGY



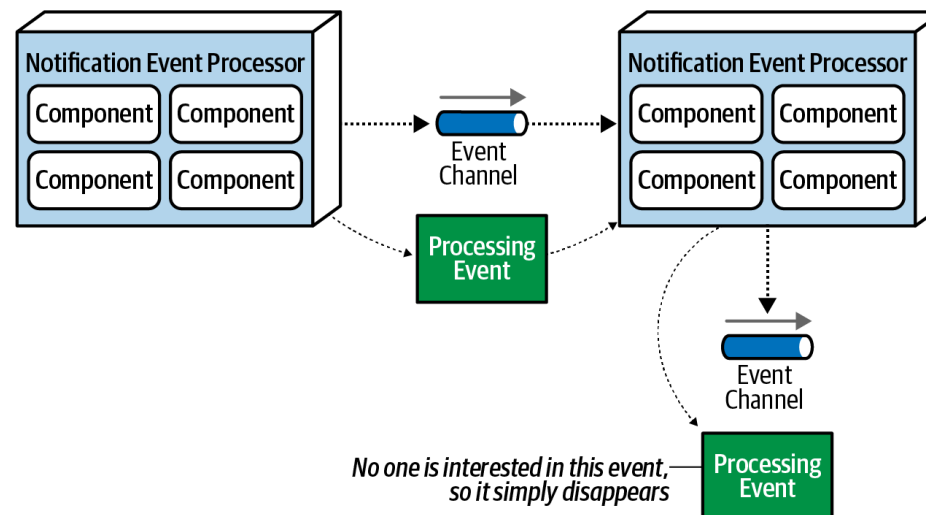
BROKER TOPOLOGY



- the **initiating event**
 - sent to an event channel in the event broker for processing
- the **event broker**,
- the **event processor**
 - Accepts the initiating event from the event broker and begins the processing
 - After performing the task it asynchronously advertises what it did to the rest of the system by creating a processing event
- the **processing event**
 - is asynchronously sent to the event broker for further processing

EVENT BROKER

- Usually multiple domain-based instances (federated)
- Each broker contains all of the event channels used within the event flow for that particular domain
- Decoupled asynchronous broadcasting approach => uses **topics based on publisher-subscriber messaging model**
- Good practice: each event processor to advertise what it did to the rest of the system

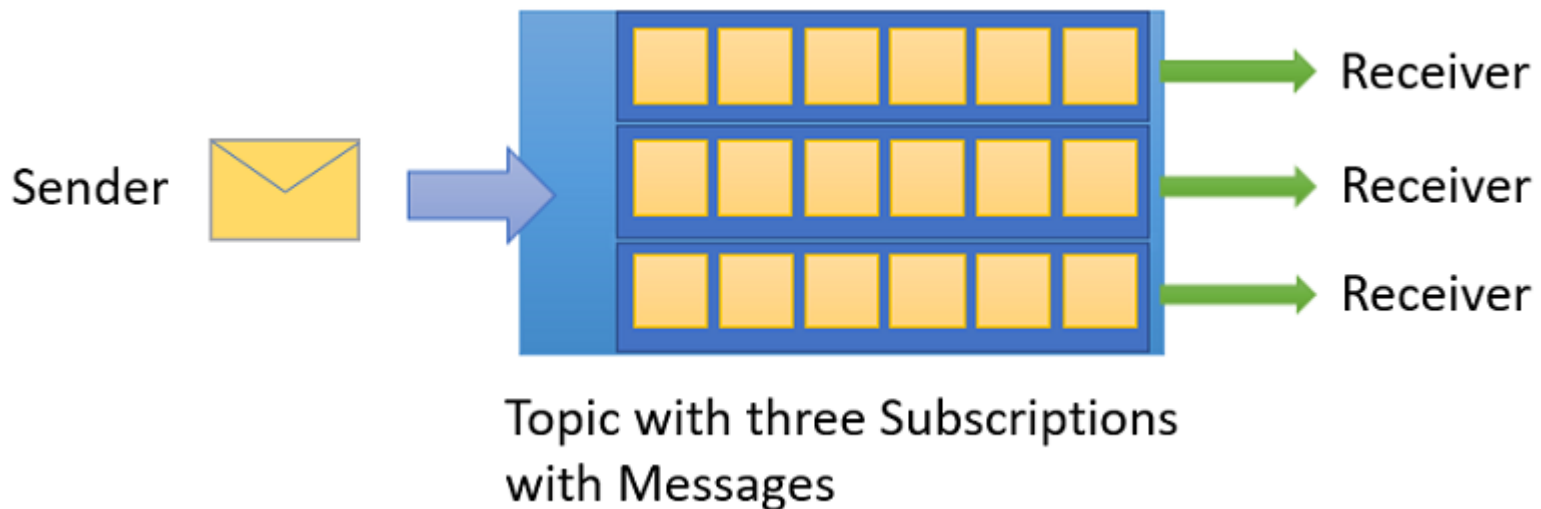


QUEUES VS TOPICS

- Queues



- Topics and subscriptions

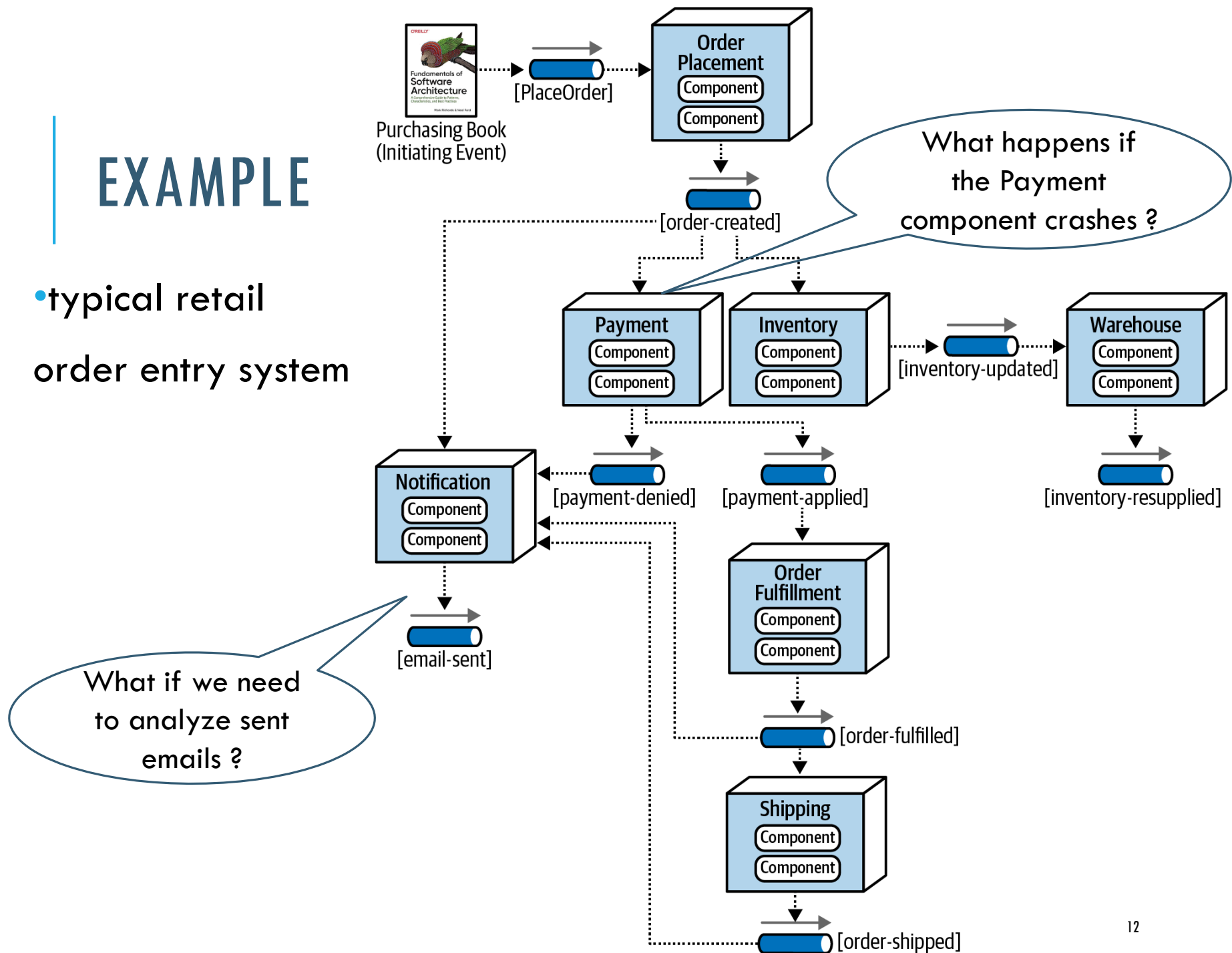


QUEUES VS TOPICS

Feature	Queues	Topics and subscriptions
Communication pattern	Point-to-point	Publish/subscribe (one-to-many)
Message delivery	Single consumer per message	Multiple subscribers can receive copies
Use case	Task distribution, load leveling	Event broadcasting, fan-out scenarios
Filtering	Not supported	Subscription filters for selective message delivery

EXAMPLE

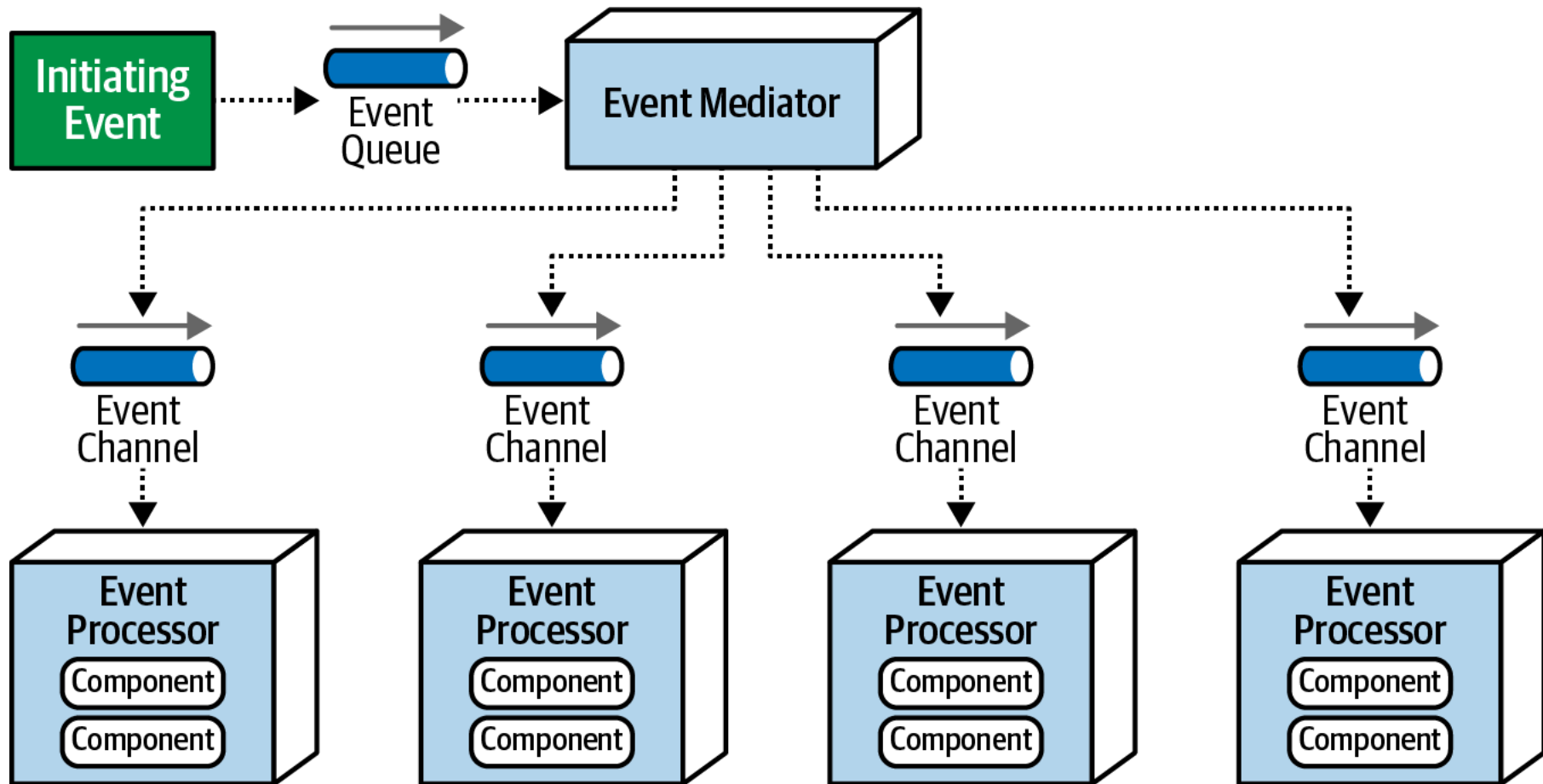
- typical retail order entry system



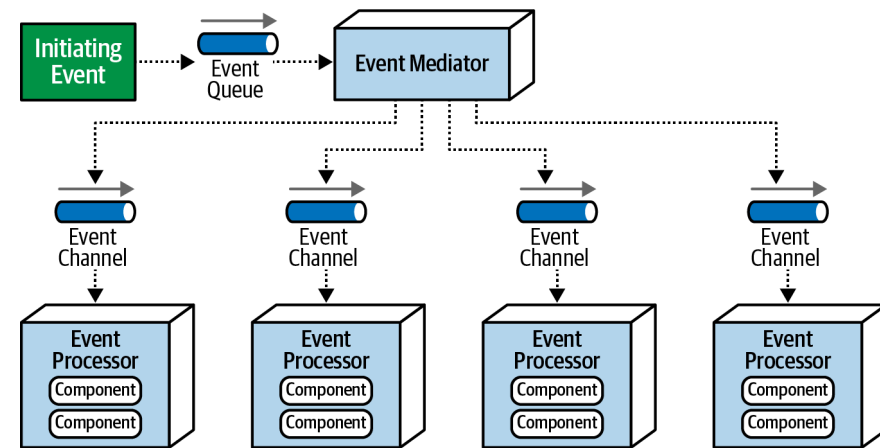
BROKER TOPOLOGY DISCUSSION

Advantages	Disadvantages
Highly decoupled event processors	Workflow control
High scalability	Error handling
High responsiveness	Recoverability
High performance	Restart capabilities
High fault tolerance	Data inconsistency

MEDIATOR TOPOLOGY



MEDIATOR TOPOLOGY



- **Initiating event:** sent to an **event queue**
- **Event queue:** accepted by the event mediator
- **Event mediator:** manages and controls the workflow => coordinates multiple event processors
 - Generates processing events sent to dedicated event channels
- **Event channels:** point-to-point messaging
- **Event processors:**
 - Listen to channels
 - Process events
 - Respond back to the **mediator**

DISCUSSION

- multiple mediators, usually associated with a domain/related events

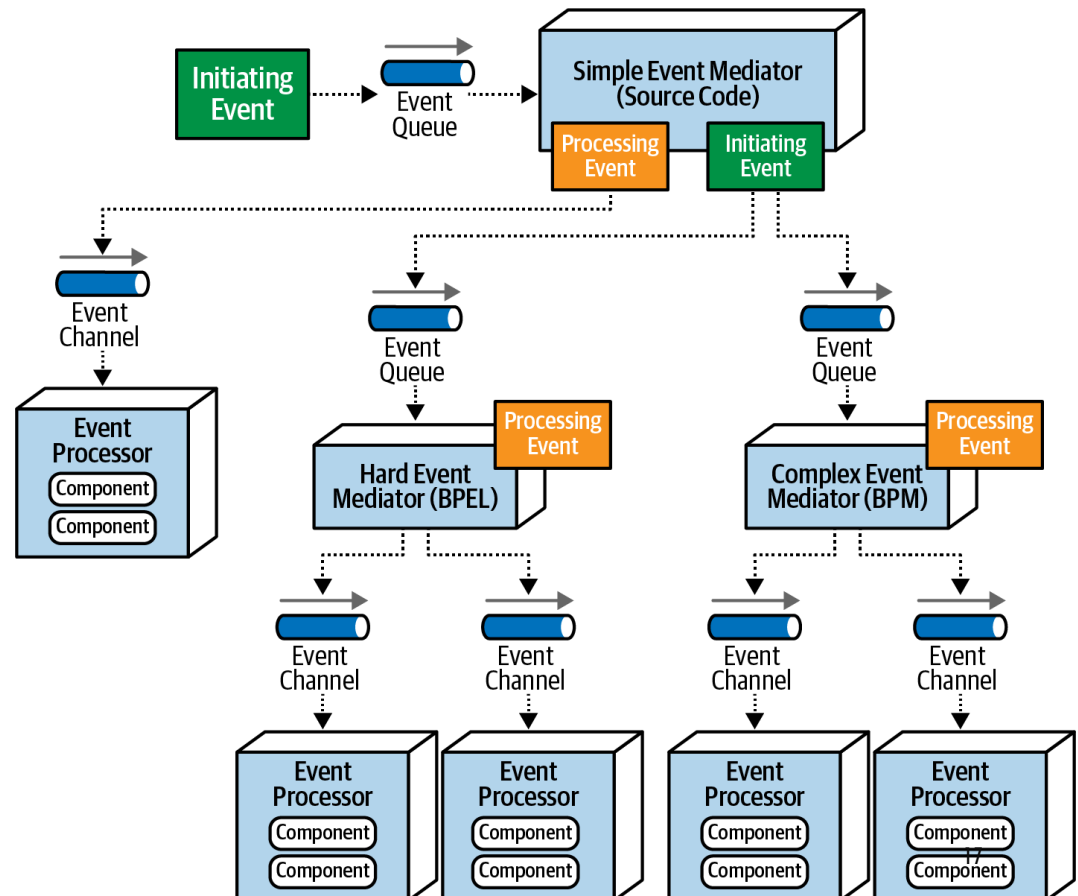
⇒ Increased reliability, overall throughput and performance

- can be implemented in a variety of ways
 - for events requiring **simple error handling and orchestration** ⇒ Apache Camel, Mule ESB, or Spring Integration (Message flows and message routes within these types of mediators are typically custom written in programming code)
 - if the event workflow requires **lots of conditional processing and multiple dynamic paths with complex error handling directives** ⇒ Apache ODE or the Oracle BPEL Process Manager (based on Business Process Execution Language BPEL)
 - BPEL does not work well for those event workflows requiring long-running transactions involving human intervention throughout the event process ⇒ Business Process Management (BPM engines)

EVENTS OF DIFFERENT COMPLEXITIES

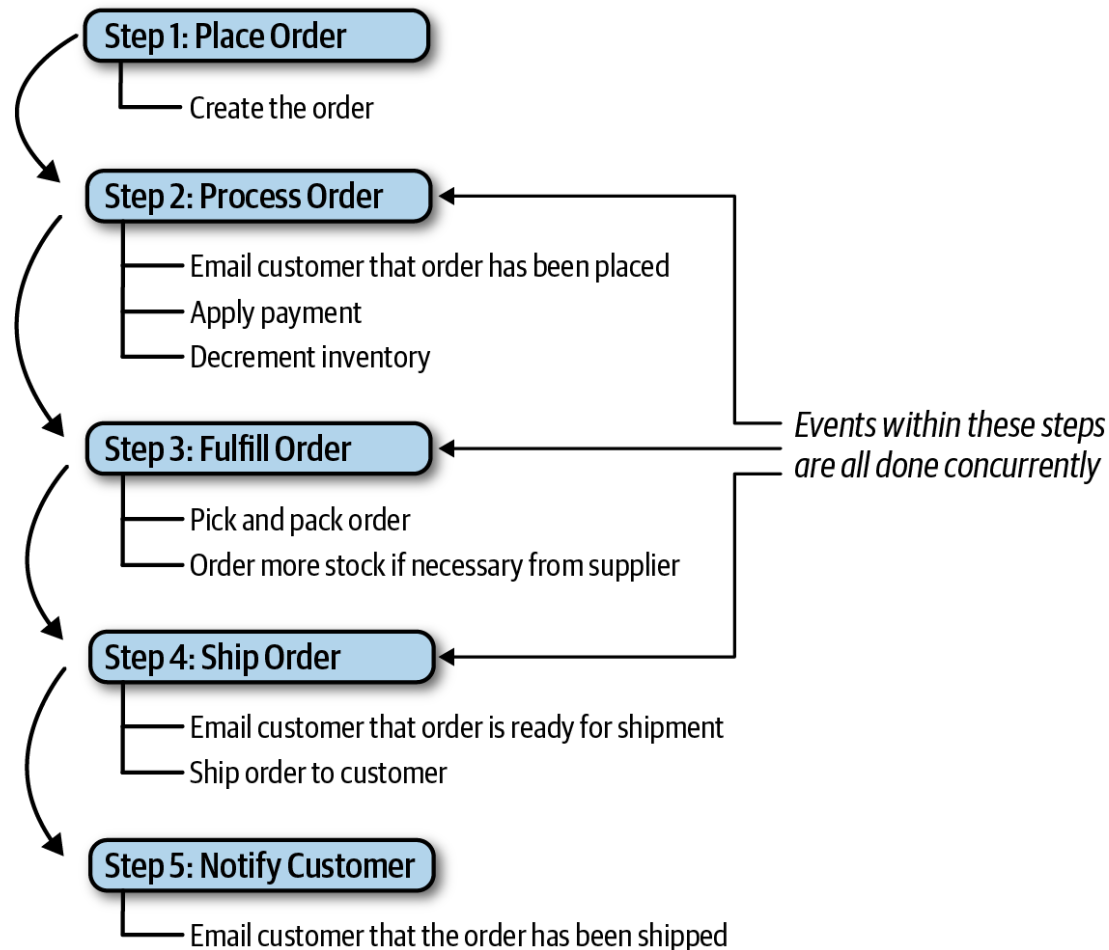
- What if you have events of different complexities? How is decided what type of mediator to use?

=> Delegation model

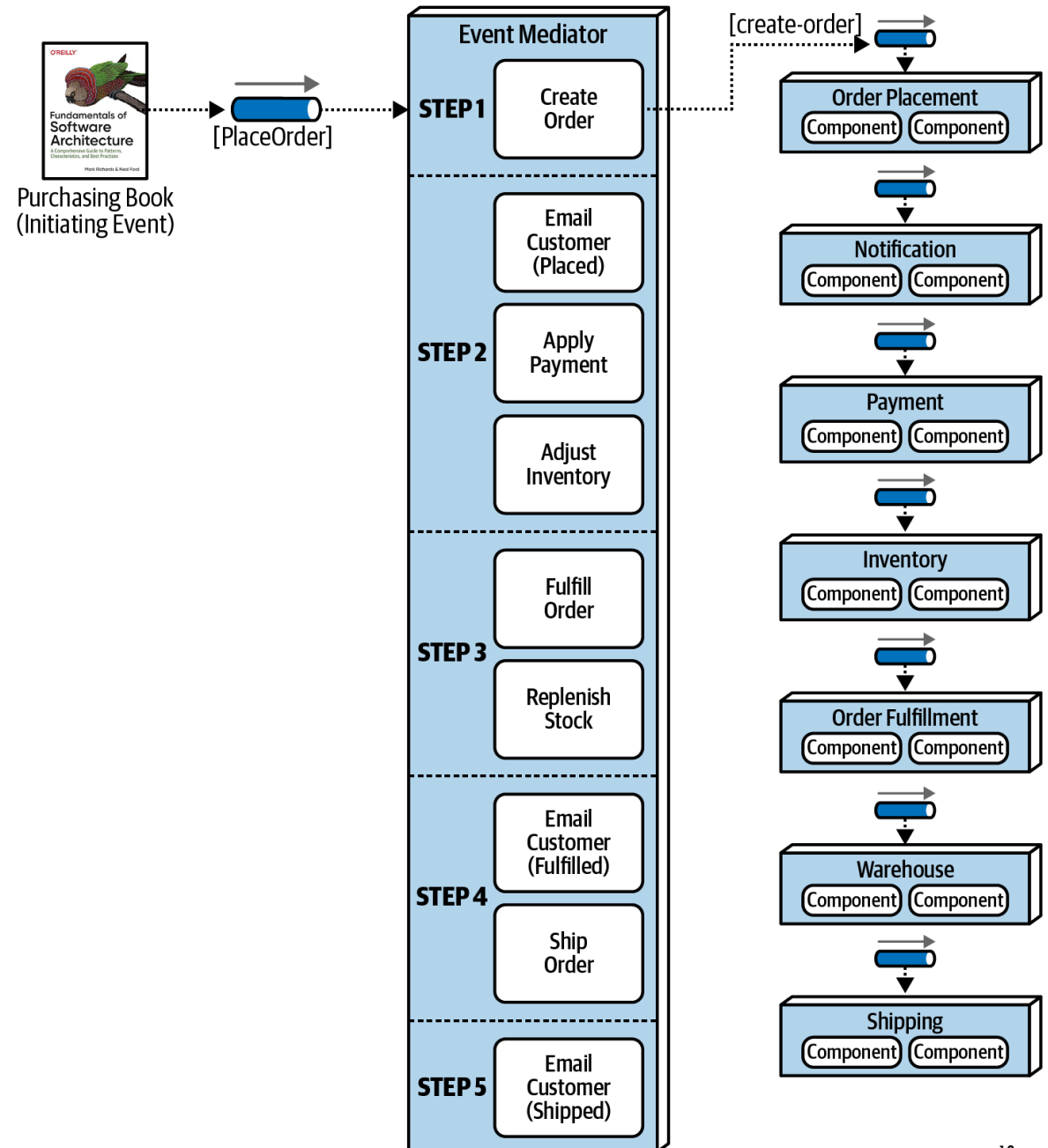


EXAMPLE

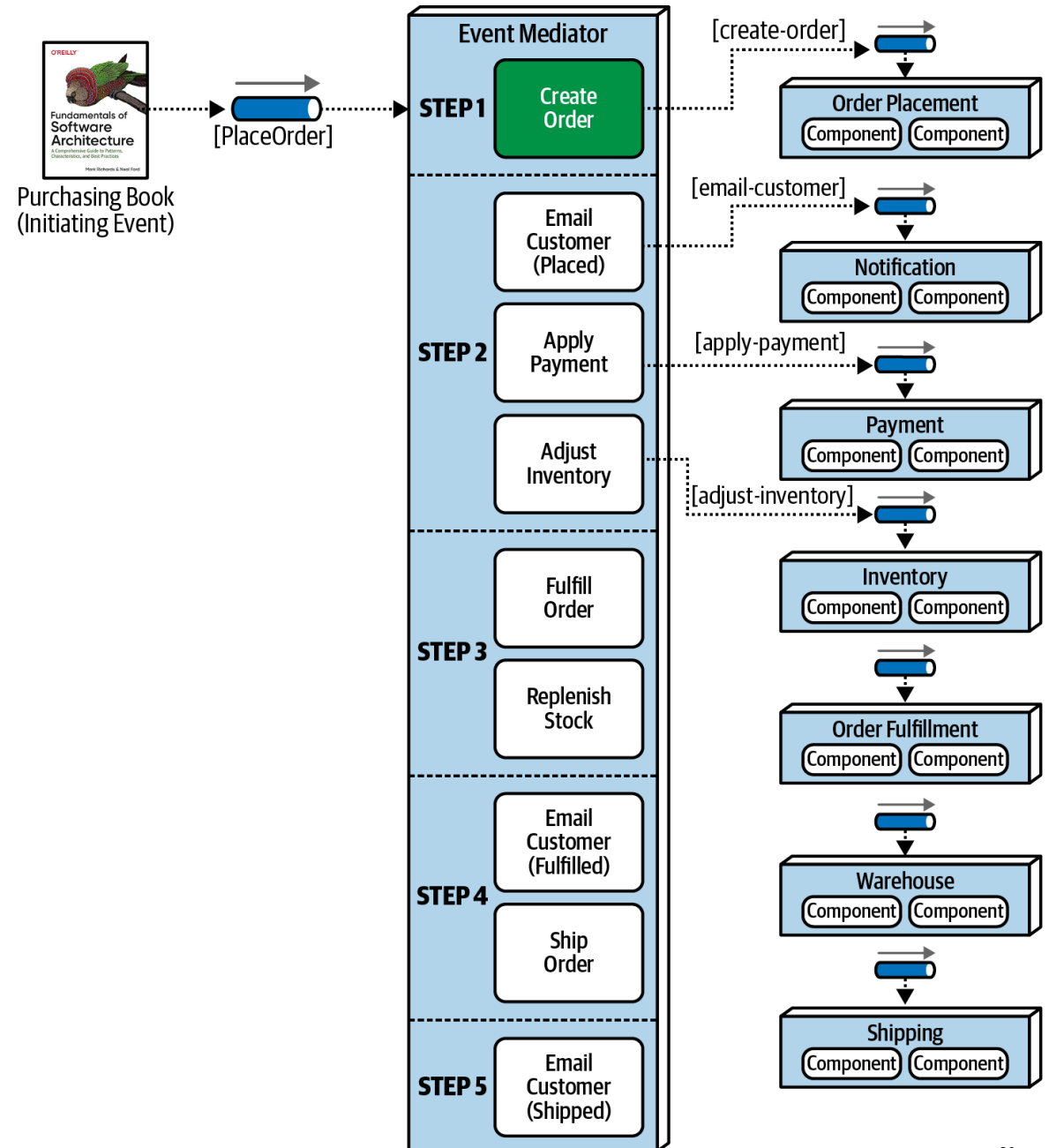
- Same retail order entry system, processing steps within the mediator



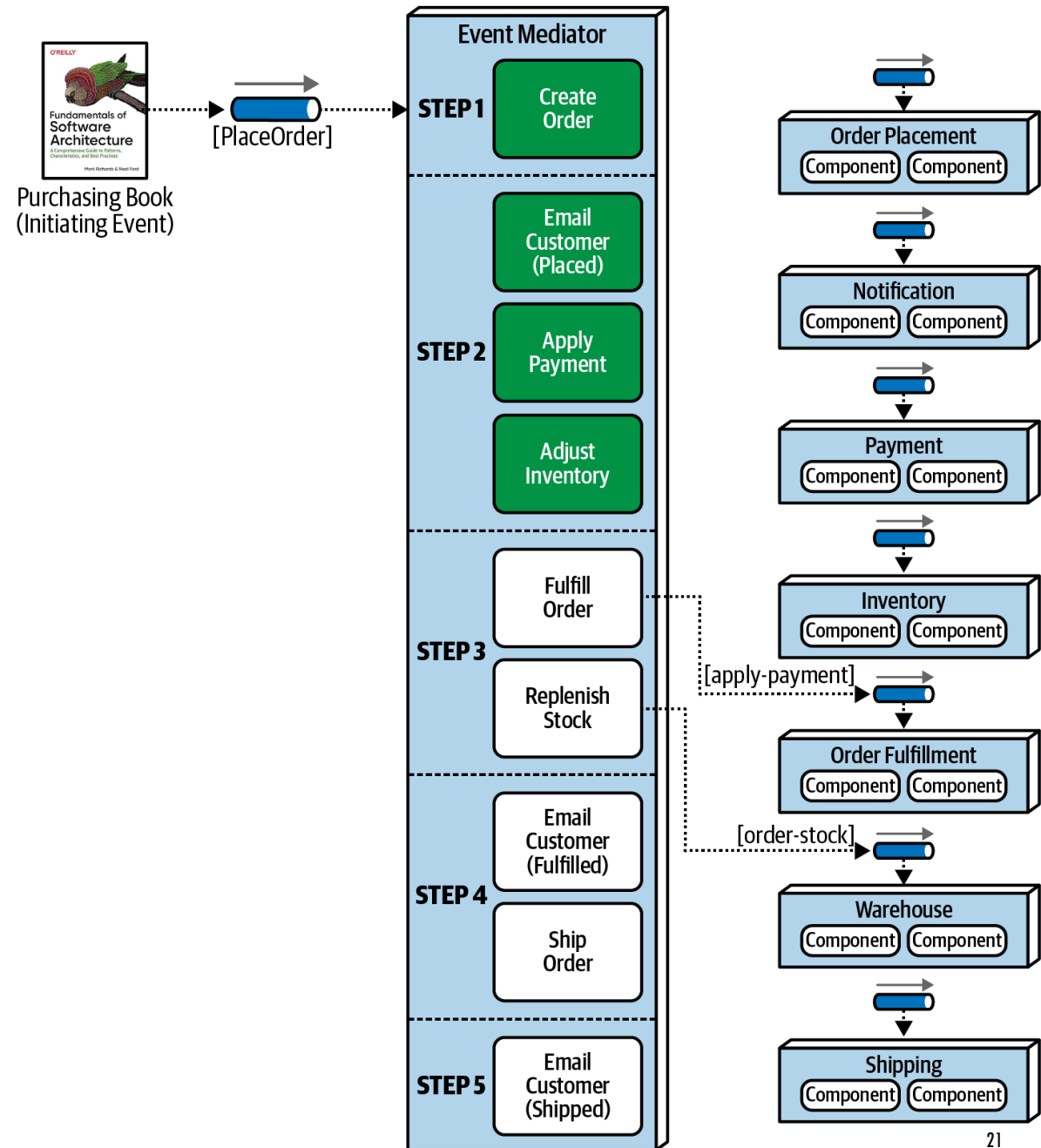
STEP 1



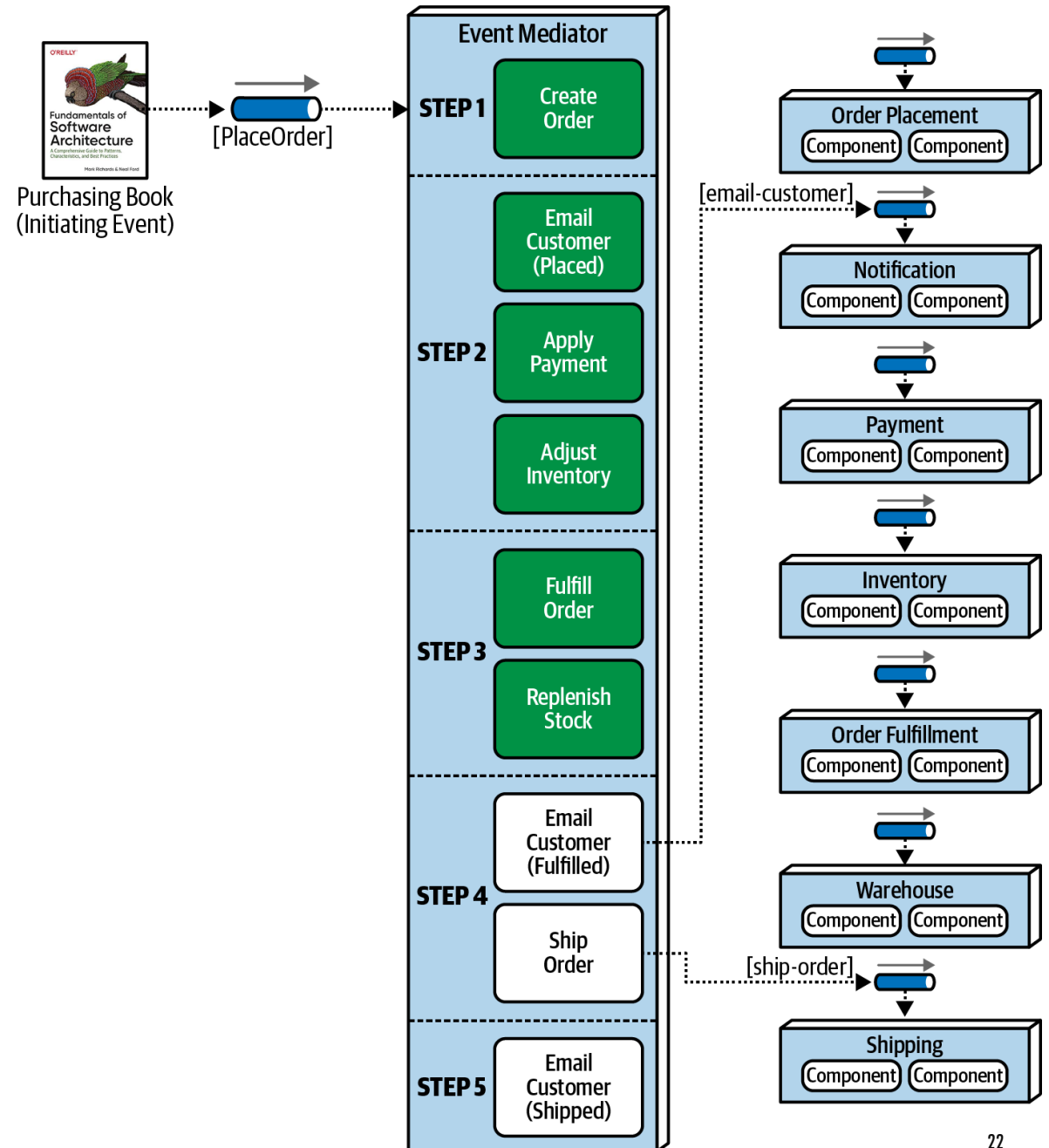
STEP 2



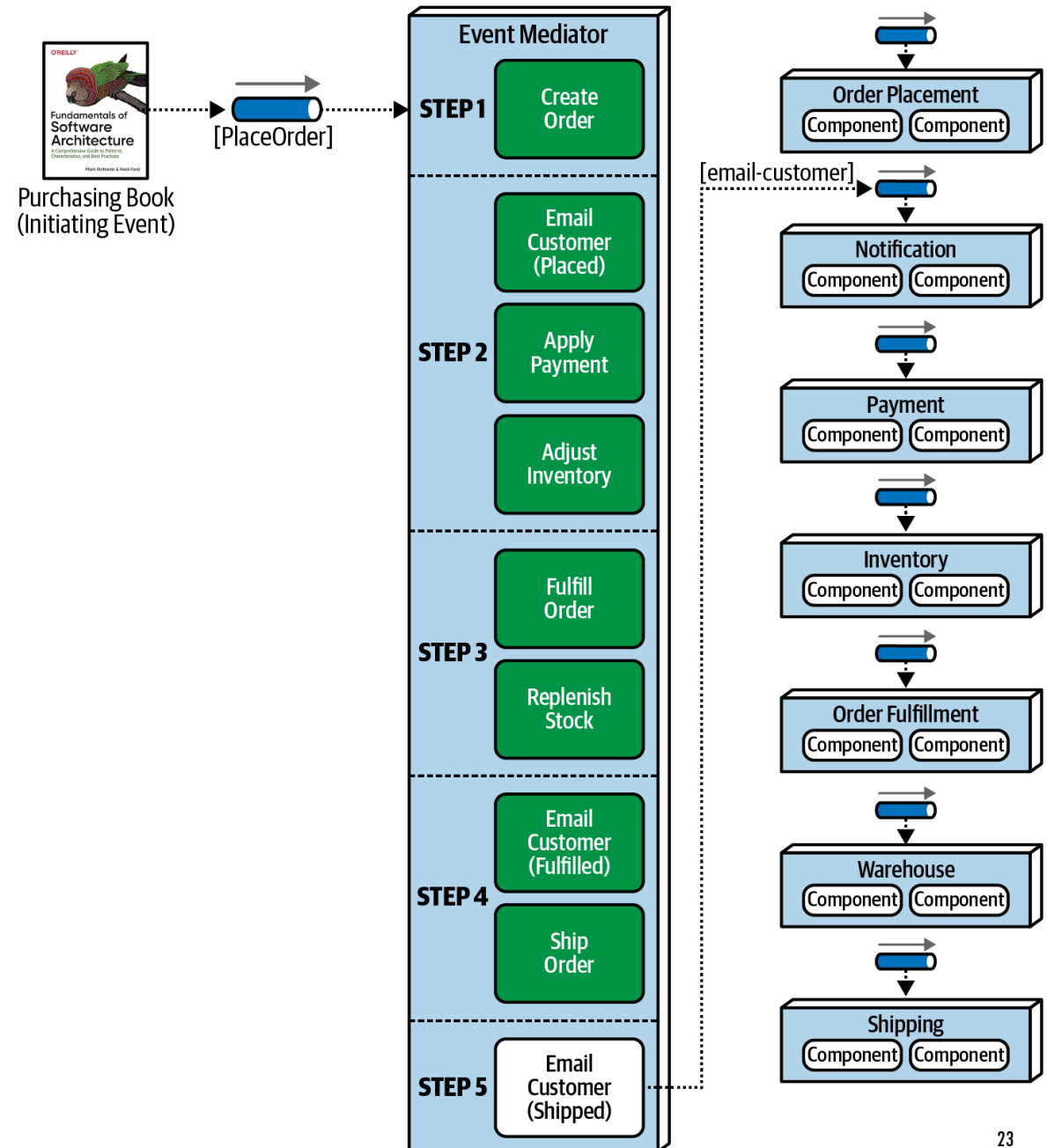
STEP 3



STEP 4



STEP 5



MEDIATOR VS BROKER

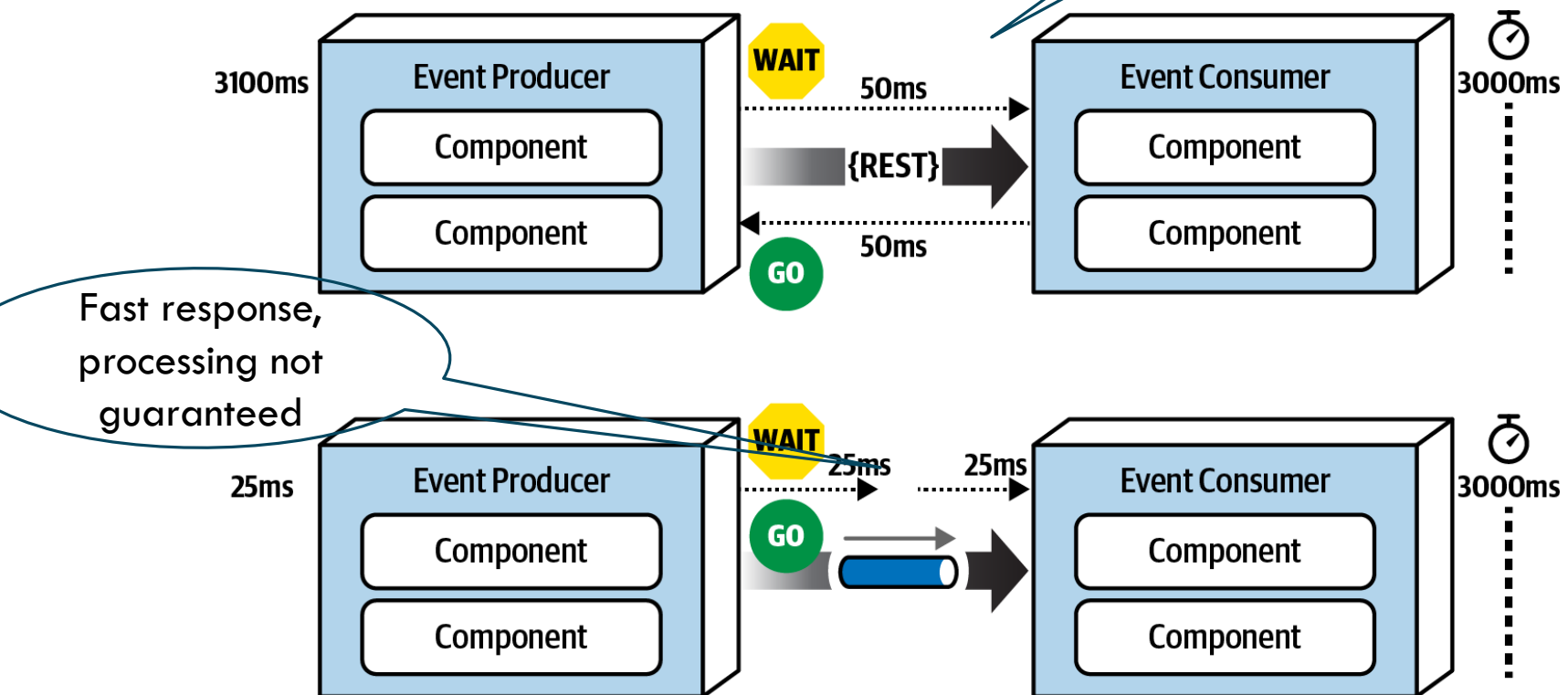
- Knowledge and control over the workflow => can maintain event state and manage error handling, recoverability, and restart capabilities
- Events are commands (**before they happen**) in Mediator (ex. place-order, send-email, etc.) => need to be processed
- Events are processes that **already happened** in Broker (ex. order-created, email-sent, etc.) => can be ignored
- **Limitations** of mediator
 - very difficult to declaratively model the dynamic processing that occurs within a complex event flow
 - although the event processors can easily scale, the mediator must scale as well, => a bottleneck in the overall event processing flow
 - event processors are not as highly decoupled in the mediator topology as with the broker topology
 - performance is not as good due to the mediator controlling the processing of the event

TRADE-OFFS OF MEDIATOR

Advantages	Disadvantages
Workflow control	More coupling of event processors
Error handling	Lower scalability
Recoverability	Lower performance
Restart capabilities	Lower fault tolerance
Better data consistency	Modeling complex workflows

ASYNCHRONOUS CAPABILITIES

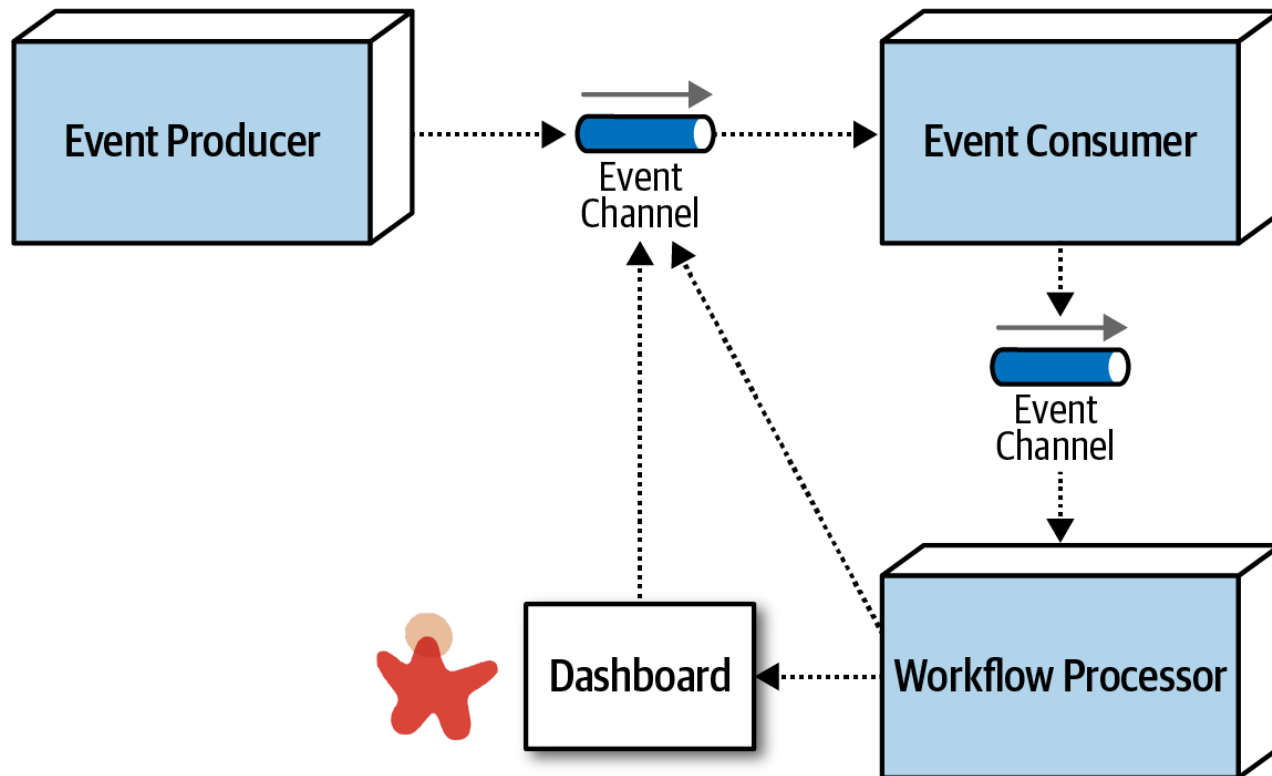
- Event-driven architecture relies on **asynchronous** communication!
- Difference between responsiveness and performance (latency)



ERROR HANDLING

- What happens in an asynchronous communication if one processing step fails?

⇒ Reactive architecture (Workflow event pattern)



WORKFLOW EVENT PATTERN

- The event producer **asynchronously passes data** through a message channel to the event consumer
- If the event consumer **experiences an error** while processing the data, it immediately delegates that error to the workflow processor and **moves on to the next message** in the event queue
- the workflow processor **programmatically** (without human intervention) **makes changes** to the original data to try and repair it, and then **sends it back** to the originating queue.
- The event consumer sees this message as a new one and tries to **process** it again
- If the workflow processor cannot fix the message => it sends it to the dashboard of human

EXAMPLE

- trade orders processing

```
Exception in thread "main" java.lang.NumberFormatException:
    For input string: "8756 SHARES"
    at java.lang.NumberFormatException.forInputString
        (NumberFormatException.java:65)
    at java.lang.Long.parseLong(Long.java:589)
    at java.lang.Long.<init>(Long.java:965)
    at
trading.TradePlacement.execute(TradePlacement.java:23)
    at trading.TradePlacement.main(TradePlacement.java:29)
```

2WE35HF6DHF,BUY,AAPL,8756 SHARES

764980974R2,BUY,AAPL,1211

1533G658HD8,BUY,AAPL,2654

EXAMPLE

- **Delegation to the Trade Placement Error service**

Trade Placed: 12654A87FR4,BUY,AAPL,1254

Trade Placed: 87R54E3068U,BUY,AAPL,3122

Trade Placed: 6R4NB7609JJ,BUY,AAPL,5433

Error Placing Trade: "2WE35HF6DHF,BUY,AAPL,8756
SHARES"

Sending to trade error processor <-- delegate the
error fixing and move on

Trade Placed: 764980974R2,BUY,AAPL,1211

...

EXAMPLE

- Trade Placement Error service deletes the word SHARES and resubmits the trade for reprocessing:

Received Trade Order Error: 2WE35HF6DHF,BUY,AAPL,8756 SHARES

Trade fixed: 2WE35HF6DHF,BUY,AAPL,8756

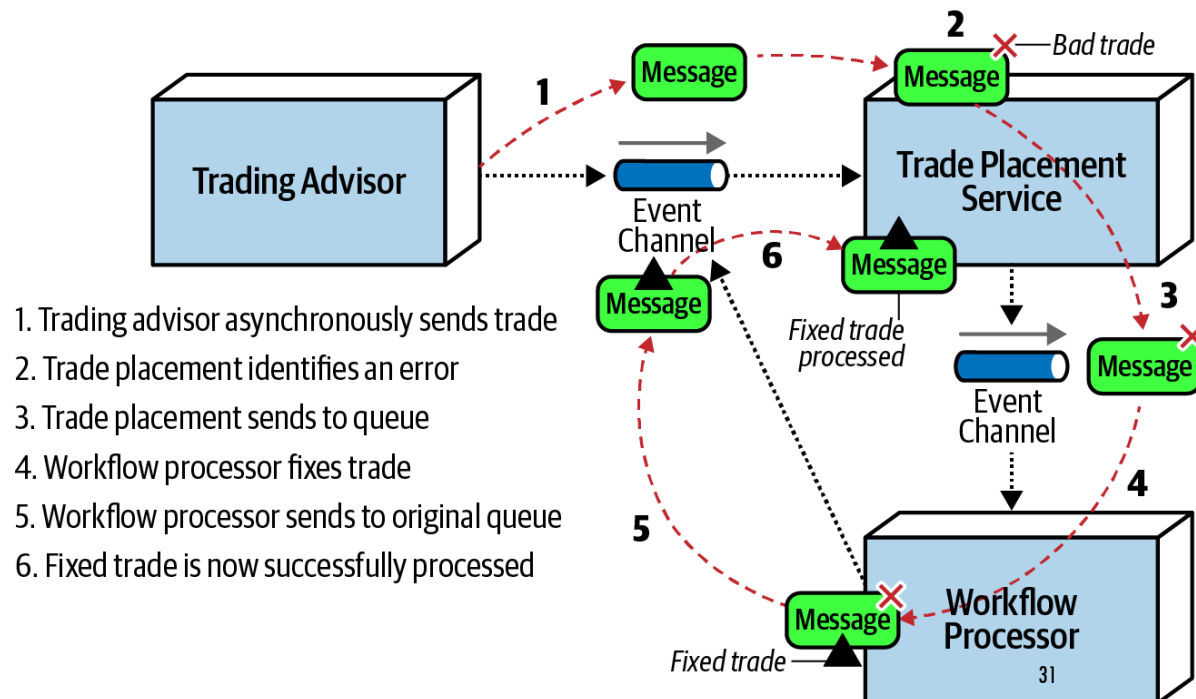
Resubmitting Trade For Re-Processing

⇒ messages in error are processed

out of sequence when they are

resubmitted!

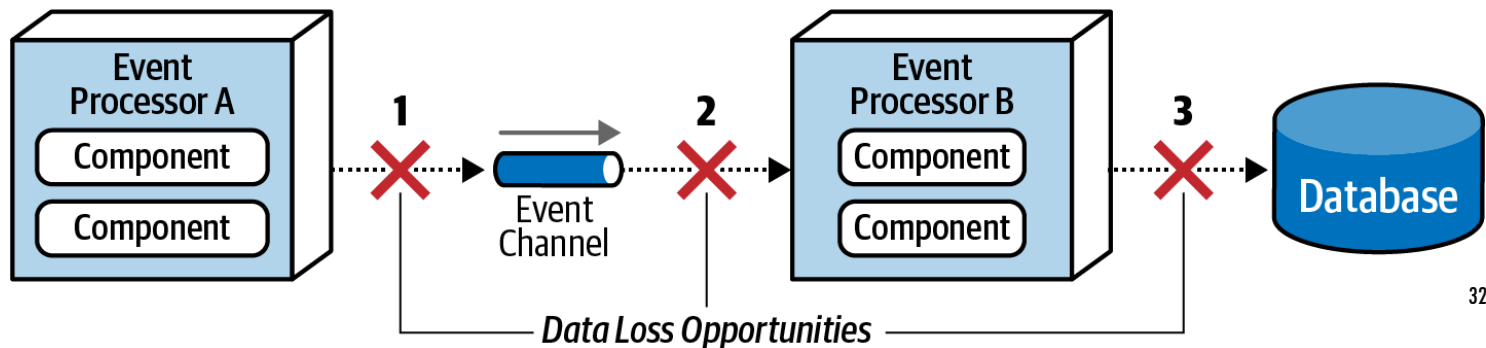
Homework! Figure out a solution!



DATA LOSS

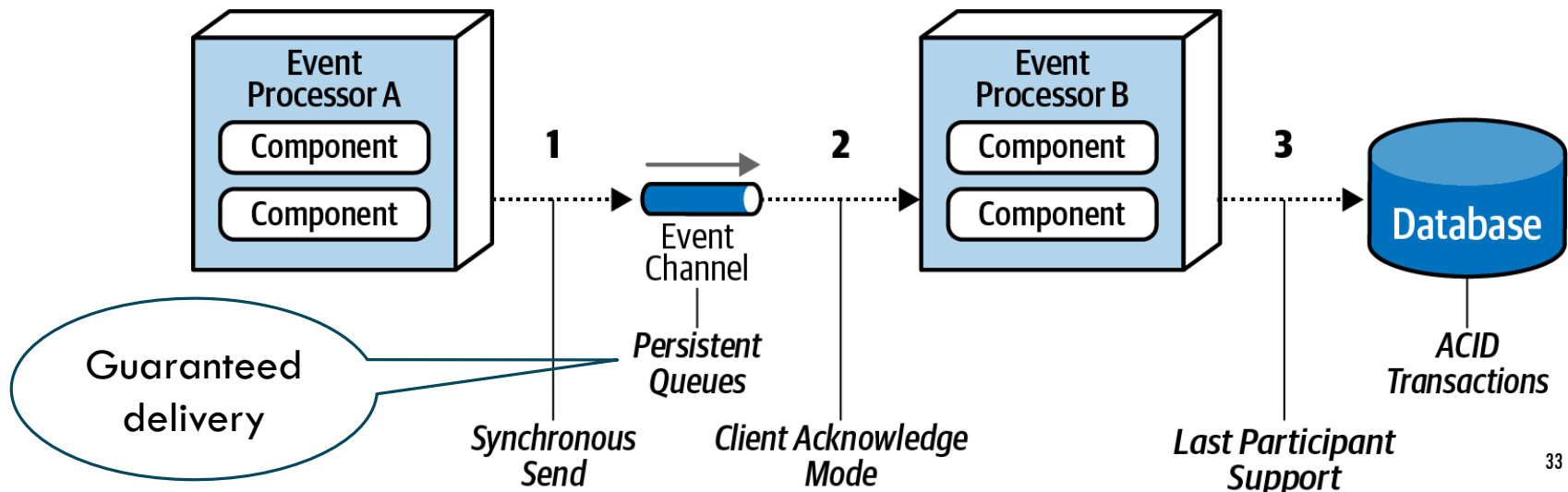
Data loss = a message getting dropped or never reaching its final destination

1. The message never reaches the queue from Event Processor A; or even if it does, the broker goes down before the next event processor can retrieve the message.
2. Event Processor B de-queues the next available message and crashes before it can process the event.
3. Event Processor B is unable to persist the message to the database due to some data error.



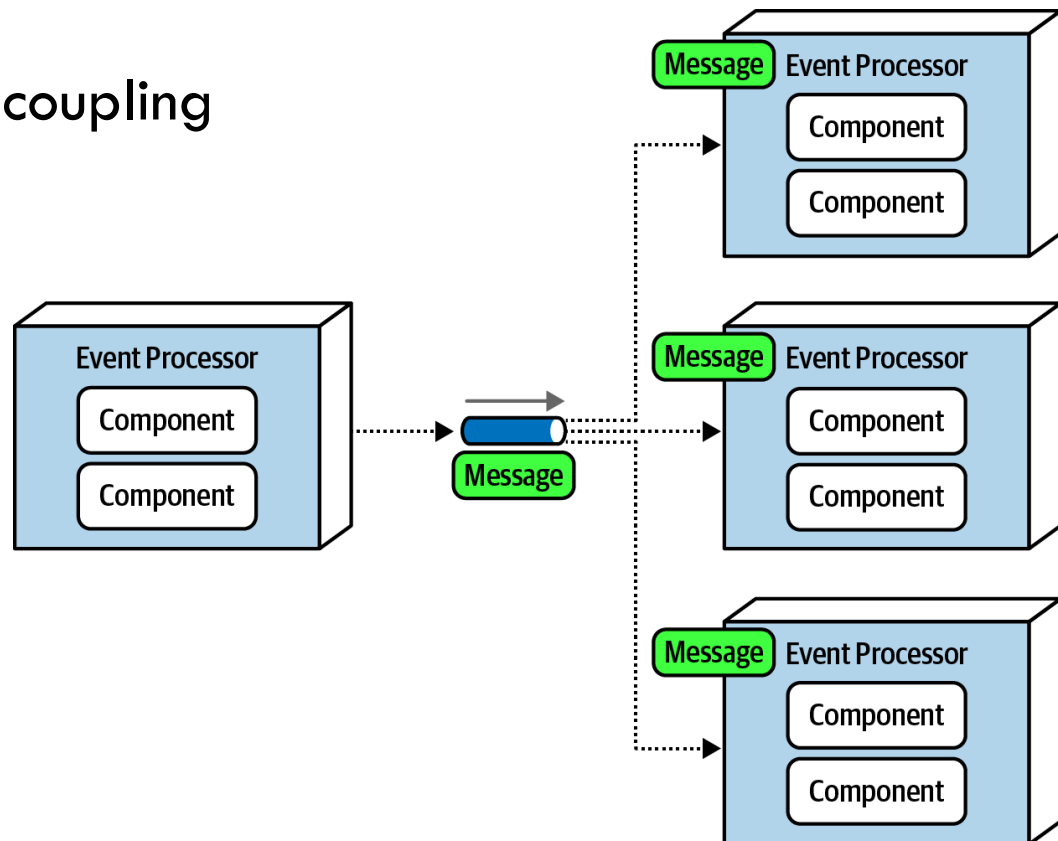
PREVENTING DATA LOSS

1. Persistent message queues + synchronous send => the message is either still with the message producer or persisted within the queue
2. Client acknowledge mode = keeps the message in the queue and attaches the client ID to the message so that no other consumers can read the message
3. ACID transactions + last participant support (LPS) = removes the message from the persisted queue by acknowledging that processing has been completed and that the message has been persisted



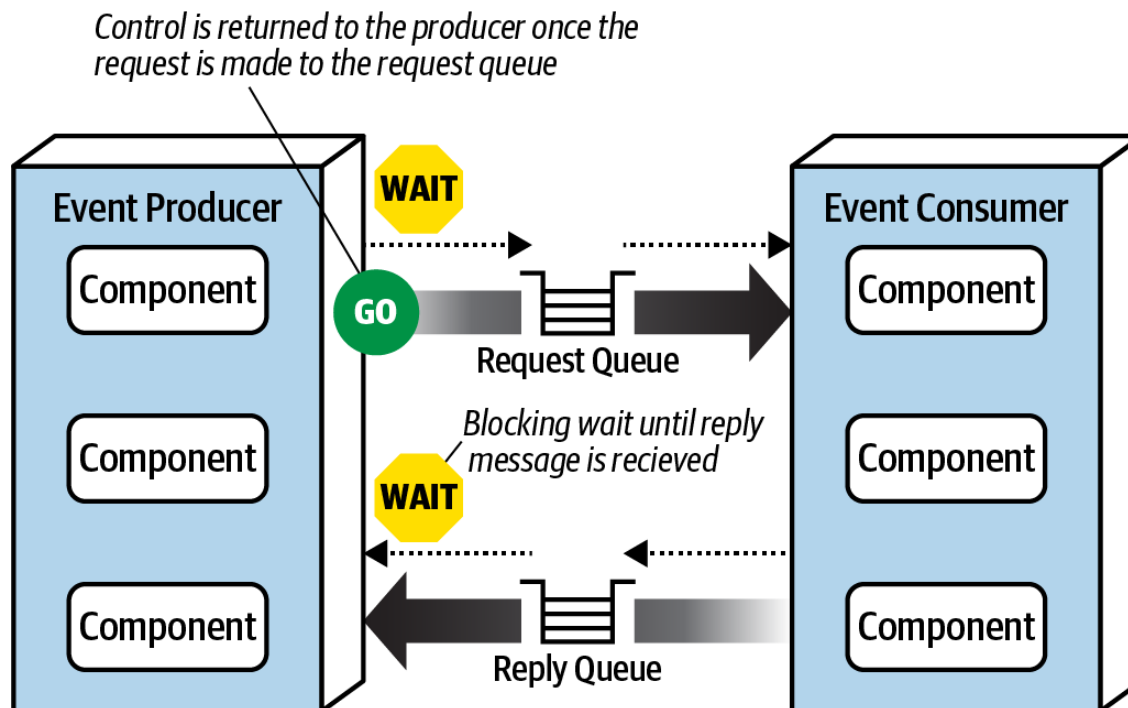
BROADCAST CAPABILITIES

- Capability to broadcast events without knowledge of who (if anyone) is receiving the message and what they do with it (i.e. publish-subscribe)
- Highest level of decoupling



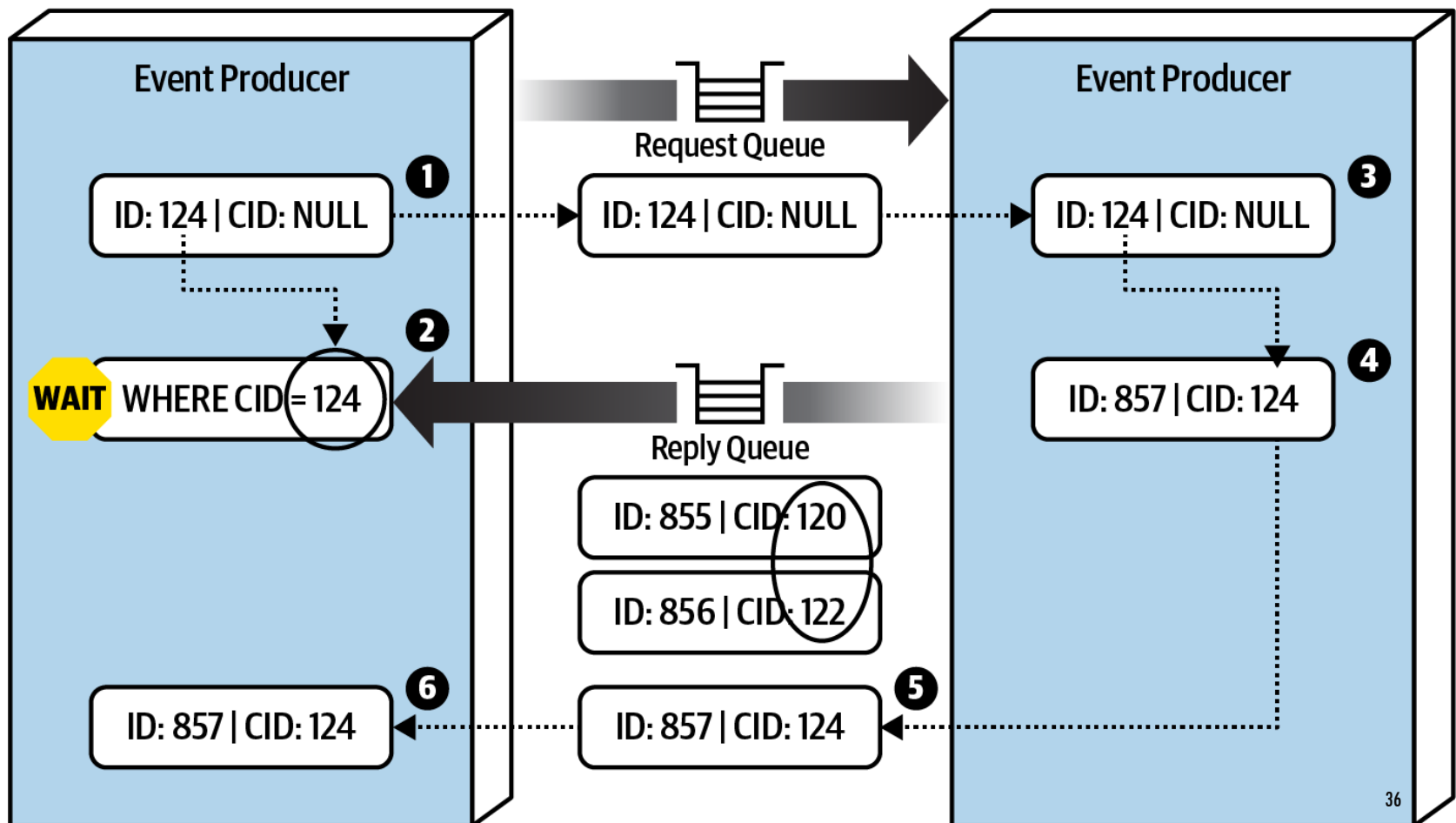
REQUEST-REPLY

- Synchronous communication is accomplished through request-reply messaging => Each event channel within request-reply messaging consists of **two queues**: a request queue and a reply queue



REQUEST-REPLY IMPLEMENTATION TECHNIQUES

- Use a **correlation ID**

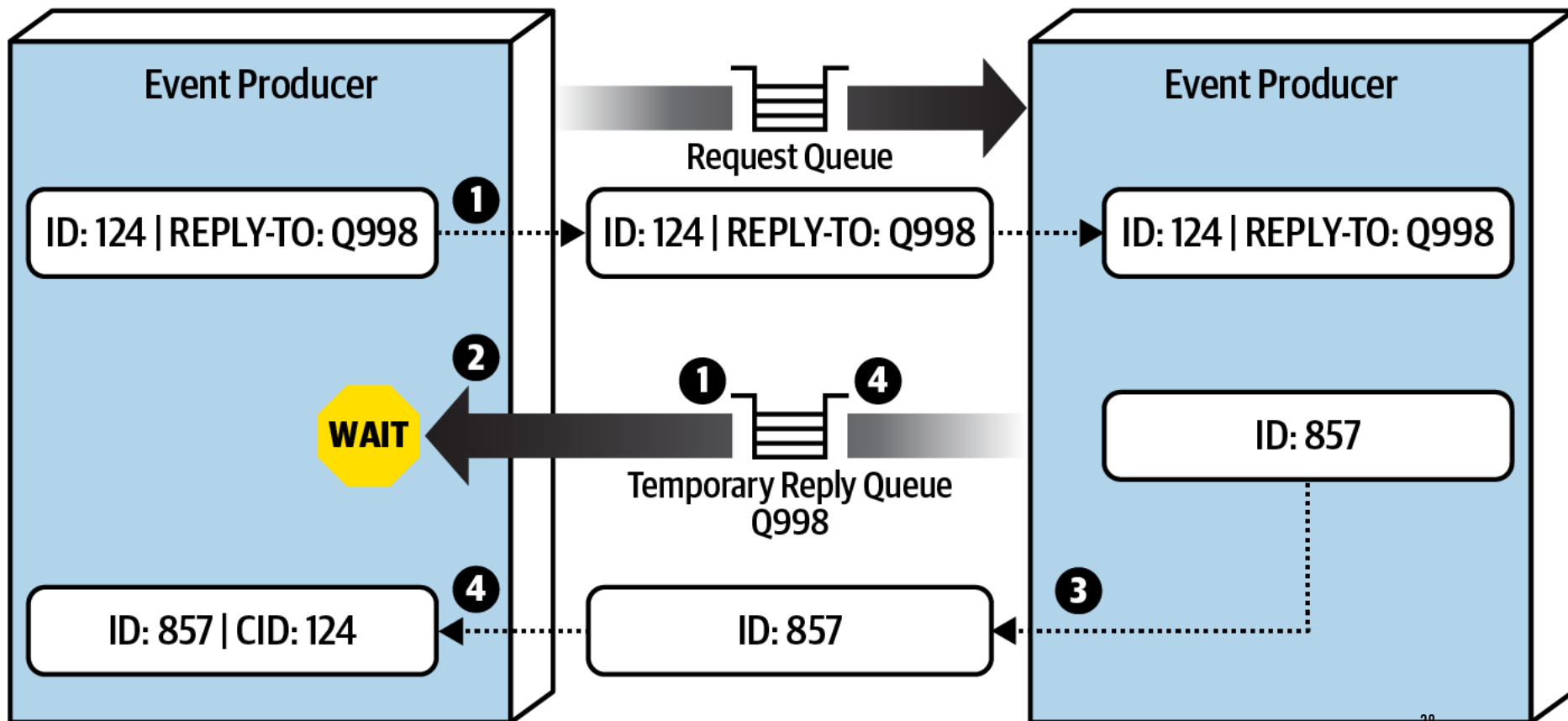


REQUEST-REPLY IMPLEMENTATION TECHNIQUES

1. The event producer sends a message to the request queue and records the unique message ID (in this case ID 124).
2. The event producer now does a blocking wait on the reply queue with a message filter (aka a message selector), where the correlation ID in the message header equals the original message ID (in this case 124).
3. The event consumer receives the message (ID 124) and processes the request.
4. The event consumer creates the reply message containing the response and sets the correlation ID (CID) in the message header to the original message ID (124).
5. The event consumer sends the new message (ID 857) to the reply queue.
6. The event producer receives the message because the correlation ID (124) matches the message selector from step 2.

REQUEST-REPLY IMPLEMENTATION TECHNIQUES

- Use a **temporary queue**



REQUEST-REPLY IMPLEMENTATION TECHNIQUES

1. The event producer creates a temporary queue (or one is automatically created, depending on the message broker) and sends a message to the request queue, passing the name of the temporary queue in the reply-to header.
2. The event producer does a blocking wait on the temporary reply queue.
3. The event consumer receives the message, processes the request, and sends a response message to the reply queue named in the reply-to header.
4. The event processor receives the message and deletes the temporary queue.

REQUEST-BASED VS EVENT-BASED

- well-structured, data-driven requests (such as retrieving customer profile data) when certainty and control over the workflow is needed => request based
- flexible, action-based events that require high levels of responsiveness and scale, with complex and dynamic user processing => event-based

Advantages of event-based	Trade-offs
Better response to dynamic user content	Only supports eventual consistency
Better scalability and elasticity	Less control over processing flow
Better agility and change management	Less certainty over outcome of event flow
Better adaptability and extensibility	Difficult to test and debug
Better responsiveness and performance	
Better real-time decision making	
Better reaction to situational awareness	

HYBRID EVENT-DRIVEN ARCHITECTURES

- Hybrid architecture = event-driven + another architectural style (ex. microservices, microkernel, pipeline)

⇒ remove bottlenecks

⇒ provides an improved level of user responsiveness

⇒ programmatic scalability to services in a microservices architecture

ARCHITECTURE CHARACTERISTICS RATINGS

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1 to many
Deployability	★ ★ ★
Elasticity	★ ★ ★
Evolutionary	★ ★ ★ ★ ★
Fault tolerance	★ ★ ★ ★ ★
Modularity	★ ★ ★ ★
Overall cost	★ ★ ★
Performance	★ ★ ★ ★ ★
Reliability	★ ★ ★
Scalability	★ ★ ★ ★ ★
Simplicity	★
Testability	★ ★

WRAP-UP

- Event-driven architectures rely on asynchronous communication between event processors
- Broker enables high performance and scalability
- Mediator enables workflow control and error handling capability
- Issues to be considered:
 - Error handling
 - Data loss
 - Broadcasting
 - Synchronous communication (Request-reply)

LET'S SWITCH TO MOODLE AND TAKE A QUIZ

It takes 4 minutes.